

INDIVIDUAL ASSIGNMENT TASK 1

TECHNOLOGY PARK MALAYSIA

CT119-3-3-VAPT
VULNERABILITY ASSESSMENT AND PENETRATION TESTING

APD3F2601CS(CYB)

HAND OUT DATE: 16th MARCH 2026

HAND IN DATE: 20th April 2026

WEIGHTAGE: 30%

NAME: HEMANTHRAJAH A/L N KRISHNARAJAH

TP NUMBER: TP070844

INSTRUCTIONS TO CANDIDATES:

1. **Assignment is to be submitted through online submission (Moodle).**
2. **Students are advised to underpin their answers with the use of references (cited using the Harvard Name System of Referencing).**
3. **Late submission will be awarded zero (0) unless Extenuating Circumstances (EC) are uphelp.**
4. **Caes of plagiarism will be penalized.**
5. **You must obtain 50% overall to pass this module.**

Table of Contents

1.0 Introduction 4

1.1 Target System 5

1.1.1 Target System and Its Purpose 5

1.1.2 Operating System 7

1.1.3 Software and Services 11

1.1.4 Legal Compliance 13

1.1.5 Chosen Type of Penetration Testing. 14

1.1.6 Penetration Testing and The Importance of Penetration Testing. 15

2.0 Scope and Application Technologies 17

3.0 Vulnerability Assessment 19

3.1 Vulnerability Assessment Methodology 20

4.0 Tools 24

4.1 Nessus 25

4.2 OWASP ZAP (Zed Attack Proxy) 26

4.3 Comparison Table between Nessus & OWASP ZAP 28

5.0 Methodology 28

5.1 NIST SP 800-115 Methodology 29

6.0 Exploitation and Attack 31

6.1 Penetration Testing Attack Method. 32

6.2 Tools 34

6.2.1 Burp Suite 34

6.2.2 OWASP ZAP 36

6.2.3 Comparison Table Between Burp Suite and OWASP ZAP 37

7.0 Conclusion 39

8.0 References 40

1.0 Introduction

One of the most feasible methods of knowing how a web application would fail under hostile conditions is vulnerability assessment and penetration testing (VAPT). In contemporary organizations, most especially those that are customer-oriented and thus require customer-facing portals, the question is not whether a vulnerability somewhere in the stack is present, but whether or not the organization can find, rank and mitigate that vulnerability before it is exploited. A web application may be business-functionally complete and insecure due to the common occurrence of vulnerabilities in input processing, session processing, role isolation, third-party relationships, server configuration and operational processing. More recent studies have indicated that testing of web applications requires both automated discovery and manual validation since no single scanner can comprehend completely the business logic, privilege limits or workflow-specific instances of abuse (Altulaihan et al., 2023; Abdulghaffar, Elmrabit and Yousefi, 2023).

The present report builds up Task 1 of the VAPT assignment, suggesting a systematic evaluation methodology of the target system of interest, Attendize, an open-source ticketing and event management platform using Laravel. Attendize enables organizers to build an event, issue types of tickets, manage attendees and accept ticket-related orders through a web-based, centralized system. In business terms, the application is very pertinent to the modern web security practice since it integrates multiple risk-related capabilities within a single platform: publicly facing event pages, organizer dashboards, checkout operations, ticket holder management, order communications and administrative features.

The proposal is reinforced by the practical environment that was ready to carry out this assignment. They were installed in an Ubuntu virtual machine and tested using a separate Kali Linux machine. The target was vulnerable at 192.168.29.130 on port 8000 which provided a controlled internal lab environment where discovery, vulnerability scanning and manual validation could be conducted safely. This is important since an effective VAPT proposal cannot be divorced of the reality of the deployment of the target. The choice of a tool, scope, methodology and anticipated risks all rely on the operating environment (Akbar, Nur Isnaini and Putranto, 2025).

Recent web application security thought supports the necessity of testing the target in a stratified manner. Broken access control continues to rank high on the list of risk factors by

OWASP Top 10:2025, and software supply chain failures are becoming a significant concern to current applications that rely on the presence of large volumes of third-party packages and libraries (OWASP, 2025a; OWASP, 2025b; OWASP, 2025c). This is particularly important when it comes to Attendize since it operates on an older Laravel/PHP stack and has numerous dependencies both on the server and the client. Such a target cannot be evaluated only based on classical injection or cross-site scripting; it must also be evaluated on the grounds of weak headers, old components, fragile business logic, workflow failures, information disclosure and misconfigured operations.

This Task 1 document is not thus an attempt to describe tools in vacuo. It aims to support a sensible VAPT approach to a deployed web application. Nessus Essentials is handy in detecting host based and configuration-based problems. OWASP ZAP can be used to spider, passively analyze and discover web-based vulnerabilities. The best option in manual interception, replay and workflow verification is Burp Suite. These tools are combined to give breadth, depth and practicality in accordance with the assignment brief.

1.1 Target System

Here, the target is described in an organized manner. It specifies what Attendize is, the environment where it was installed, the software and services that comprise its attack surface, the legal and ethical boundaries of the test, the type of testing that was chosen and the extent of the proposed engagement. This failure is significant since the quality of a VAPT proposal lies in the ability of the target to be described in a way that the methodology and tool selection can be made coherent.

1.1.1 Target System and Its Purpose

The chosen target system in this report is Attendize, open-source ticketing and event management software, based on Laravel, and created with the aim to facilitate event management end-to-end. As its description on the public repository states, Attendize is supposed to aid organizers in creating events, publishing types of tickets, managing attendees and accepting ticket-related orders through a centralized web-based application (Attendize, 2024). This is significant since the target is not a training platform that has been deliberately made vulnerable or a static Web site. It is a commercial app that simulates the type of workflows that are predominant in

contemporary customer-facing systems: account registration, data submission, event publication, transactional workflow and administrative management.

Attendize has a simple purpose as far as business is concerned. It enables an organizer to organize an event, establish classes of tickets, control sales dates, gather customer and attendee data, manage fulfilment processes including order confirmation and event check-in. This implies that the platform has to be able to host both the general users and privileged users. The event details, purchase of tickets and checkout should be made available to the general users. Organizers require event control, attendee management and operational controls on a dashboard level. The multi-role implementation greatly enhances the security relevance of the application since the system is not merely storing content; the system is making trust decisions on many levels.

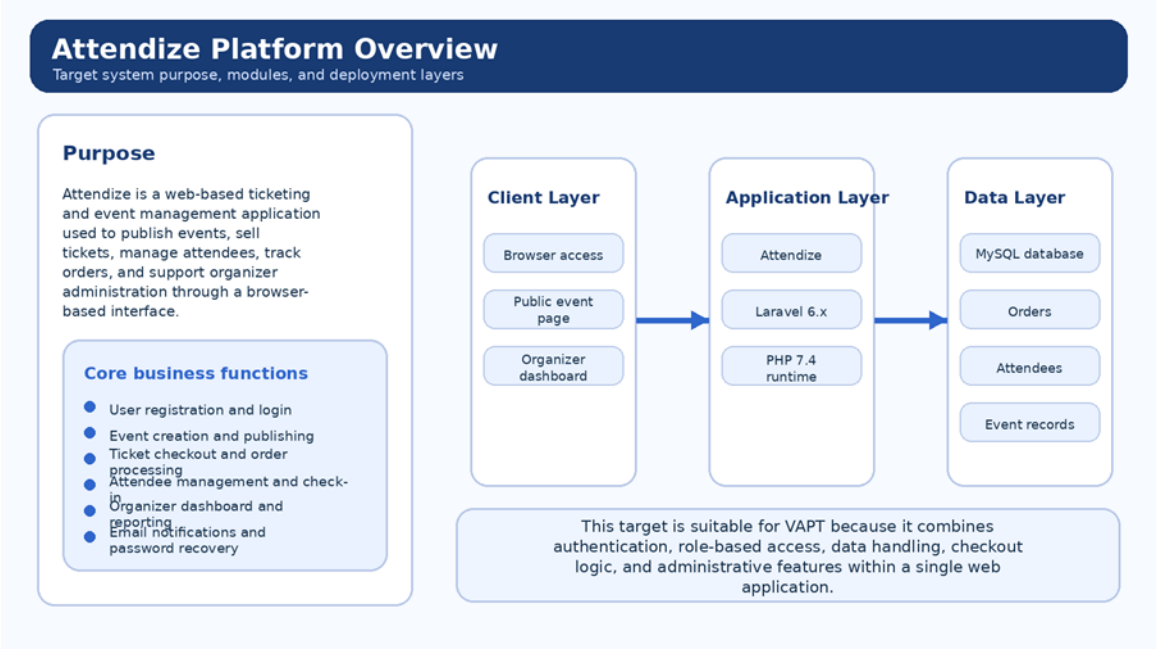


Figure 1: Attendize platform overview

The target is thus an excellent academic option since it reveals most of the areas of risks as outlined by modern web security advice. According to the 2025 OWASP Top 10, broken access control, insecure design, cryptographic failures, software supply chain failures and security logging failures are some of the most crucial web risks of the present time (OWASP, 2025a). Attendize is a helpful case study since all those issues can be reasonably traced to the features of real applications. Such as, order data and attendee information provide confidentiality risk,

organizer actions provide authorization risk, checkout and ticket state provide business-logic risk, and the dependency stack presents component and supply-chain issues.

A second reason the target is suitable is that the platform is reflective of much of small-to-medium organizational deployments. Business applications based on open-source often use a combination of defaults provided in the frameworks, community-packaged applications, custom controllers and third-party client-side libraries. The latest research indicates that such a combination may result in an uneven security maturity, with some of the layers being highly secured and the rest of the layers being vulnerable to weak defaults, legacy dependencies or insufficient validation logic (Altulaihan et al., 2023; Gandikota et al., 2023). It is that mixture of business relative, technical scope and lab reality that makes Attendize an appropriate target system to use in this VAPT proposal.

1.1.2 Operating System



Figure 2: Ubuntu Logo

This assignment deployed its target environment based on Ubuntu in a virtual machine. Ubuntu was chosen as the server-side operating system as it is commonly used to deploy web applications, has excellent package ecosystem support, easily interoperates with Apache, PHP and MySQL and is available in a form that is easily reproducible in an academic lab. The Attendize application was deployed in the real-world configuration on the Ubuntu virtual machine and made

available to the testing machine through the local network at 192.168.29.130 on port 8000. This was sufficient to make the environment realistic enough to discover hosts, scan and intercept requests but safely not interfere with production services.



Figure 3: Laravel logo

The platform was also technically appropriate to support a Laravel-based application with legacy dependency requirements using Ubuntu. In setup, it was apparent that the application could not be run on the original newer environment of PHP and instead needed a PHP 7.4-compatible setup due to the limitations of its own packages and age. The operating system is significant to the proposal since it does not merely run the application; it determines the compatibility and patching strategy as well as the surface of exposure of the entire stack. As a web application relies on an older runtime, the operating system is included in the security conversation. The server can be stable, yet it faces a maintenance dilemma as well since an older application dependency can be unresponsive to modernization (Laravel, 2026; OWASP, 2025c).

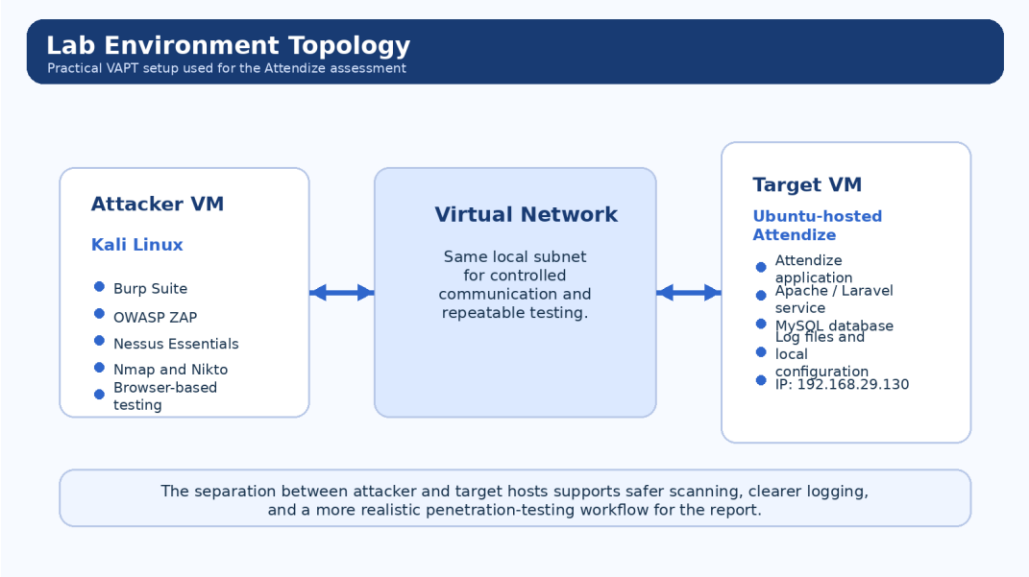


Figure 4: proposed VAPT lab

The attacker environment for this proposal was separated from the target. The assessment workstation was a Kali Linux system that was used to launch Nessus Essentials via its local web interface, to run OWASP ZAP, to run Burp Suite, and to browse the target in a controlled offensive-testing environment. The separation of this attacker/target is good practice since it represents a more realistic pattern of engagement and minimizes the possibility of polluting the application with superfluous development tooling. It further facilitates better recording of network boundaries which is a plus to the proposal and any subsequent exploitation report. Ubuntu host is the application server, and Kali is the system of the tester.

On a VAPT planning level, the operating system is important due to a variety of reasons. First, it has an effect on the services that are visible to a scanner. A Linux host can be vulnerable to exposing Apache version data, HTTP method response, robots.txt, and network-level metadata in a manner that can be determined by Nessus. Second, it has an impact on logging and troubleshooting. In practical testing, the logs of Laravel which were stored in the Ubuntu file system came in handy to get a feel of the root of the failure of checkouts. Third, it influences the variety of hardening measures, which might be subsequently prescribed, such as firewall policy, ICMP behavior, reduction of version disclosure, package maintenance and isolation of service.

Ubuntu is also pertinent since it encourages the use of lightweight, repeatable lab infrastructure. When it comes to an educational VAPT proposal, a target should be not only

testable but controllable as well. Ubuntu enables the tester to install or reconfigure Apache, PHP extensions, MySQL, Composer and other services in a flash. It helps in easy access to application logs, environment variables and artisan commands as well. This facilitates repeat of problems, test fixes and speculates over the reaction of infrastructure decisions with application behavior. Such value of reproducibility in a report such as this is because it implies that the proposed assessment can be justified as both structured and practical and not aspirational.



Figure 5: Kali Linux Logo

Lastly, the operating system also has a subtle yet significant impact on risk categorization. Other problems with the target are host or configuration level such as ICMP timestamp behavior, version disclosure or HTTP method exposure. Others are application-level but can only be observed as the server environment is accessible, like mail configuration, queue management or runtime incompatibility. Thus, the section on the operating system is not merely a formality in which the system is described. It elaborates on the basis on which the whole evaluation is based. In this Task 1 proposal, this pragmatic target can be defined with specificity as an Ubuntu-hosted Attendize deployment at 192.168.29.130:8000, which is tested by a separate Kali Linux environment, using Nessus Essentials, OWASP ZAP and Burp Suite.

Another reason why Ubuntu is the right choice is that it enables realistic troubleshooting regarding security. When working on a practical, log files, artisan commands, PHP extensions and restarts of the service were all part of the assessment. This can assist in a VAPT environment since not all risks are fully comprehensible when the tester is not able to match browser behavior with evidence on the server side. An operating system that obscures such details as a target would diminish the quality of analysis. Ubuntu on the other hand will enable the easy collection of evidence that facilitates better reporting and remediation planning.

1.1.3 Software and Services



Figure 6: Apache Logo



Figure 7: MySQL Logo

The target system is a combination of multiple layers in software and services that constitute the application attack surface. Attendize was accessed at the web layer using Apache HTTP Server and accessed at an application endpoint on port 8000 in local lab. The application relied on PHP 7.4 at the runtime layer due to the compatibility between the application and the framework and packages. At the data layer, persistent application state (event records, order data, account details and other transactional content) was stored using MySQL. To support this basic stack, there were Composer-driven PHP dependencies, browser-side JavaScript libraries, session cookies, CSRF tokens, and a mail/notification system that participated in processing orders.

On the application level, Attendize offers the basic business functionality: to create an event, to publish its tickets, to approve an order, to process a checkout and to manage it on the

organizer's side. Security-testing wise, web routes and workflows are not the most important services but may be separate daemons. The information-gathering and content-delivery surface is revealed on public event pages. Identity and session logic is revealed in the login and account routes. Checkout paths reveal input verification, state management and business process regulations. Authorization controls and possible insecure direct object reference attacks are potentially revealed by organizer routes. This is the reason why a web application evaluation should consider software functionality as a service inventory.



Figure 8: PHP Logo

This target is particularly on the dependency profile. In the setup and scanning, it was revealed that the application has an older Laravel generation and old-fashioned JavaScript libraries like jQuery 2.1.4. The 2025 update of OWASP specifically makes software supply chain failure a significant category of risk since insecure, out-of-date, or poorly managed components can increase exposure despite otherwise secure business logic (OWASP, 2025c). In practice, obsolete packages may open pathways to indirect attacks, undermine browser-side defenses, limiting runtime upgrades and making security maintenance more difficult.

The assessed software and services were chosen on the tester side, to reflect the target profile. Host-level and web-server-level information was collected with the help of Nessus Essentials. The main automated web vulnerability scanner employed was OWASP ZAP that was used to crawl, passively alert and check web-specific header/component tests. Burp Suite was used to inspect and manipulate the HTTP requests manually and particularly checkout process, those

routes with sessions and those with organizer driven actions. This tool alignment echoes the greater principle of best-practice that host scanners, DAST-style scanners and interception proxies all seek to answer distinct questions, and that integrating them together yields a more credible coverage of assessment.

Since the service is exposed over HTTP, in a lab environment, but not using a fully hardened production reverse proxy, the application is also a good chance to compare the development-like deployments with production-like deployments. This is important as a lot of student-built systems or small business systems exist in a similarly transitional state with convenience settings being left on longer than they ought to. Running Attendize as such thus enhances the authenticity of the proposal.

1.1.4 Legal Compliance

Though this assignment is done within an academic environment, adherence to legal and ethical controls is still vital to the design of VAPT proposal. Penetration testing can only be justifiable when it is approved, scoped and controlled. This is why the suggested assessment will be limited to the local installation of Attendize on a self-hosted system running on a virtualized Ubuntu system that will be owned and managed by the institution and used with educational purposes. In this proposal, no testing will be done on a third-party production system, a public cloud tenant, a real payment service or real customer information.

The initial principle of law is authorization. The target environment is a personal lab deployment that is within the control of the tester, which meets the minimum expectation that the assessment should be done on approved systems. This aligns with how contemporary studies on web penetration testing characterize the scope of acceptable scope: the tester and system owner must concur on the target, the limits, the methodology and the reporting expectations prior to commencing with attack simulation (Akbar, Nur Isnaini and Putranto, 2025; Imtias et al., 2025).

The second legal principle is data protection. In a lab, ticketing and event applications process personally identifiable information. Real data categories can be reflected in such fields like first name, last name, email address, ticket holder information and order history. Thus, the proposal includes the use of non-sensitive or synthetic test data. Images of the screenshots, logs or recorded requests must not reveal actual external credentials or unrelated personal information in future documentation.

The third is non-destructive testing. The proposed assessment will specifically exclude denial-of-service attacks, data corruption which is destructive, the installation of malware and any activity that is intended to be persistent even after the testing is over. It also does not cover attacks on actual third-party providers regardless of the application having outbound email or payment integrations. Lastly, the proposal presupposes the transparent reporting: the findings will be categorized, rated by their severity, described in business terms and provided with remediation guidance. Legal compliance here is more comprehensive than law in the limited sense; it also involves ethical restraint, scope discipline and honest presentation of what was really evaluated.

1.1.5 Chosen Type of Penetration Testing.

The type of penetration testing that will be chosen in this proposal is gray box web application testing. This option is the best since the test is being conducted on a locally deployed application with the overall architecture the tester knows, yet still needs to perform route discovery, request interception and workflow analysis in runtime. An extreme black box would disregard the practical benefits of being aware of the target stack and would decrease efficiency during a constrained academic period. An extreme white box would exaggerate the knowledge of the tester too and push the exercise too far out of context of the real attackers. Gray-box testing offers a realistic and educational intermediate between two extremes.

This stance is supported by recent comparative work. Research comparing frameworks of web penetration testing still indicates that the choice of methods must be based on the goal of the engagement as well as the level of internal knowledge at hand (Imtias et al., 2025). The gray-box scenario allows the tester to understand well enough to concentrate on the significant application features, like check out, order handling, headers, components and organizer actions, without abandoning the necessity of actual discovery and validation. This can be especially helpful when the platform is such as Attendize since potential failure is not just in the source code, but in the behavior of the application when subjected to real HTTP interaction. A lot of problems, such as the absence of headers, brittle workflow state, dependency-related warnings and misconfigured services, can only be discovered when the application is stressed by the browser and proxy software.

Gray-box testing is also appropriate as the target was literally constructed and tested in the local lab. The tester is aware of the operating system, approximate stack, service IP and the assessment tools under use. Nevertheless, the tester is not privileged to have administrative visibility of all control paths. This renders the use of scanners to discover, proxies to intercept and review logs to confirm. The chosen kind of penetration testing is thus aligned with both the real-life situation of the deployment as well as the academic necessity of suggesting a feasible approach. It is stringent to the extent that it yields results of significance but is limited to the extent that it is focused on authorized and reproducible testing.

1.1.6 Penetration Testing and The Importance of Penetration Testing.

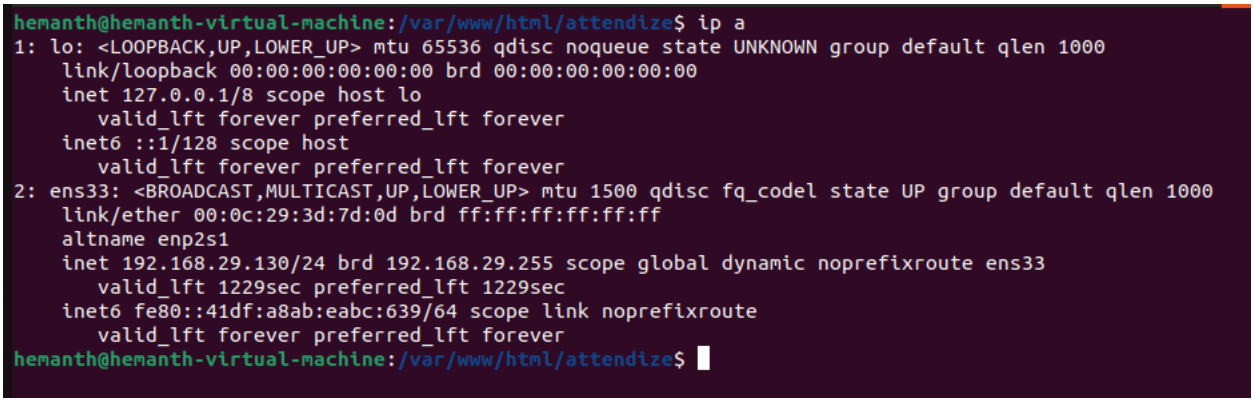


Figure 9: Ubuntu IP

This proposed assessment will be confined to the Attendize application running on the Ubuntu target host at 192.168.29.130:8000 and the software/services directly supporting the deployment. The in-scope web components are the public event pages, the login and session behavior, the checkout validation and order creation routes, the organizer-facing dashboard functions, the delivery of client-side dependencies to the browser, the response headers, the version disclosures visible in the service, and the application-linked auxiliary behavior with direct impact on order processing, such as outbound mail configuration. Host-level reconnaissance and low impact vulnerability discovery using Nessus Essentials is also included in the scope as long as the findings are immediately applicable to the exposure of the application.

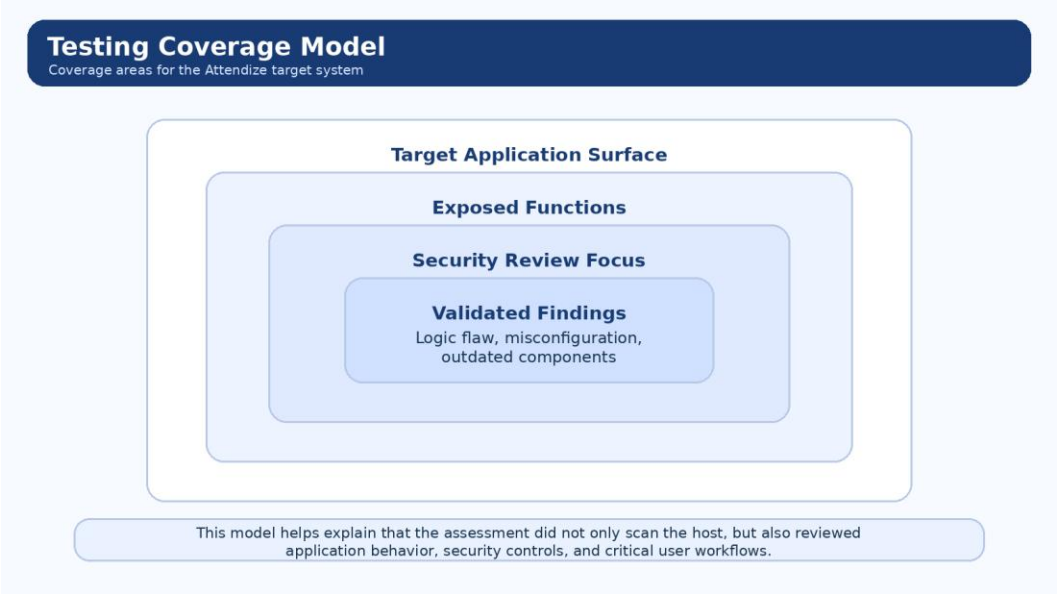


Figure 10: Testing Coverage Model

Out-of-scope: Denial of service testing that is destructive, privilege escalation testing against the underlying operating system, unauthorized brute-force testing against third-party services unrelated to the service under test, testing of real third-party providers, attempts to extract data beyond the minimum required to verify a weakness. These omissions are intentional since the assignment is on VAPT methodology and controlled exploitation and not on maximizing damage.

Penetration testing in this case is very critical since Attendize has both public-facing and privileged flows. Users post personal information, ticket purchaser information and transaction information. Organizers deal with events, ticket policies and performance. The system is thus at the nexus of confidentiality, integrity and availability. In case of mismanagement of input in the application, an unscrupulous user can interfere with the process of order creation. A low-privileged user can have an organizer-like impact in case the application fails to deal with authorization. When the application is based on old parts, the vulnerabilities are possibly more likely to be compromised. In case the application reveals excessive server details, attackers can more effectively hone their probing (OWASP, 2025a; OWASP, 2025b; OWASP, 2025c).

Special attention should be paid to broken access control since ticketing systems are role-sensitive in nature. The orders of an organizer should not be interfered with by another organizer. An unprivileged user must not be able to make state changes that are the privilege of privileged accounts. OWASP still includes broken access control as the most critical type of risk to a web

application and provides examples that are quite relevant to event systems, such as parameter manipulation and access to unauthorized account information (OWASP, 2025b). The software supply chain risk is also relevant since the actual implementation revealed that the application relies on an older runtime as well as older packages to operate. Availability is also important: in case of checkout flow failure, the event organizer will not be able to sell the tickets. All this is why penetration testing should be done to protect the integrity, availability and trustworthiness of the platform.

Penetration testing is also essential since it enhances prioritization. Not all alerts should have the same urgency of remediation as they might be created by a scanner. Using vulnerability assessment and manual testing in controlled format will enable the organization to differentiate between cosmetic hardening vulnerabilities and vulnerabilities that have a material impact on orders, attendance and organizer control. In the case of a ticketing platform, such a difference is critical since the most harmful issues tend to disrupt the confidence in the ordering process.

2.0 Scope and Application Technologies

The testing is restricted to the web application layer of the Attendize ticketing and event management system, which is locally hosted in an isolated virtual machine environment to test penetration testing in academics. The testing is based solely on what is accessible through a browser in the application and the web functionality. There will be no destructive activities in the assessment. Denial-of-service attacks, permanent database modification, record deletion, or testing against unauthorised third-party systems are all out of scope. This guarantees that the testing is ethical, repeatable and in line with the controlled requirements of the assignment.

Category	Item	Details
Target System	Application Name	Attendize
Target System	Application Purpose	Ticketing and event management
Target System	Deployment Type	Locally hosted web application in virtual lab
Target Host	Operating System	Ubuntu Linux
Target Host	IP Address	192.168.29.130
Target Host	Web Service Port	8000
Target Host	Web Server	Apache HTTP Server 2.4.58
Target Host	Scripting Environment	PHP 7.4.33

Target Host	Database Service	MySQL
Attacker Machine	Operating System	Kali Linux
Attacker Machine	Role	Security testing and exploitation platform
Tools	Vulnerability Assessment Tools	Nessus Essentials, OWASP ZAP
Tools	Enumeration Tools	Nmap, Nikto
Tools	Exploitation / Manual Testing Tool	Burp Suite
Scope	In-Scope Area	Public web application interface and related user workflows
Scope	Out-of-Scope Area	DoS, destructive attacks, unauthorized external services

Table 1 Detailed System Information

The target system in question can be reached via IP address 192.168.29.130, which is the Ubuntu host where the Attendize application will be deployed in the local lab setup. The web application that is opened in the browser is the main point of assessment, in particular, the main event and ticketing functions that are provided over the HTTP. This environment enables the testing to be performed in a safe manner, with the Kali Linux testing machine, and still retain a realistic attacker-target configuration.

Reconnaissance and scanning phases revealed the active web-related services on the target host. According to the practice testing, the service related to the assessment is:

- Port 8000 (HTTP) – Main Attendize web application interface

This service carries the application functionality core and is the primary target of security testing. The testing is hence focused on the workflows and user interactions that will be exposed via the Attendize interface that will be running on port 8000 such as event viewing, ticket selection, checkout, authentication, and dashboard-related operations.

The deployed application stack shows that Attendize is running on a PHP-based backend and an Apache web server in the Ubuntu environment. In the real-world configuration, the Attendize source code is compatible with PHP 7.4-based runtime, as the legacy dependencies also demand.

Apache is deployed to assist in the delivery of HTTP services and forwarding of the incoming browser requests to the application, whereas, PHP will have the role of processing form submissions, session management, business logic treatment, and interaction with the back-end database. The significance of this combination is related to the security aspect since vulnerabilities in the PHP-based request processing, session processing, input validation, or server architecture can have a direct impact on the security of the application.

The Attendize platform has a few exposed components, which are of interest in vulnerability assessment and penetration testing, functionally. They can be pages of public events, ticket checkout, order creation, authentication interface, and administrative or organizer-side dashboard functionality. These types of modules handle user input and execute actions on the server side, which can be made vulnerable to test vulnerabilities, like a failure to validate user input, business logic errors, insecure request processing, issues with sessions, and misconfiguration.

Particular care is also taken regarding hidden resources, exposed paths and configuration-related disclosures which can provide useful information about the internal structure of the target application. This contains files or responses that might accidentally reveal web server information, framework behaviour, application paths or other security-related information. Testing is all done under controlled lab conditions to ensure safety, reliability and ethical penetration testing practice.

3.0 Vulnerability Assessment

The systematic process of discovering weaknesses, misconfigurations and obsolete aspects that can enhance the chances or severity of compromise is termed vulnerability assessment. Vulnerability assessment in web applications should be viewed as a rigorous discovery phase and not as a complete stand-alone solution. Scanners can detect missing headers, known components, exposed paths, server banners and some classes of input vulnerability, but cannot completely understand whether a workflow is exploitable in practice. The latest research regarding web testing tools has been adamant that coverage and efficiency should be tested automatically, but relevance

and severity should be validated manually (Altulaihan et al., 2023; Abdulghaffar, Elmrabit and Yousefi, 2023).

In the case of Attendize, vulnerability assessment is particularly critical since the application is a combination of server configuration, issues with browser-facing components, and business logic complexity. The target is not a one-page site with the key questions that are restricted to banner revelation or old packages. It is a transactional system, which relies on a variety of steps, cookies, CSRFs, and internal transitions of state. Thus, the objective of vulnerability assessment in this report is two-fold: to create an evidence-based image of the application exposure surface; and second, to identify where to focus later in the process of manual exploitation testing. The choice of Nessus Essentials and OWASP ZAP was because they can provide an insight into that surface in its complementary sides.

Practically, the vulnerability assessment stage of this proposal will be set up to respond to a couple of questions. What will a scanner tell you about the host? What are the headers and version markers present? Does it have evident web hardening gaps? Are old libraries present in the client side? What are the automatically mappable routes and public assets? Does the target show indicators of software age or configuration drift that cause manual testing to have more urgency? It is not intended to purport compromise on the basis of scan output alone, but to generate a dependable prioritized map of weakness categories that merits further testing at a subsequent level.

Because of this, the vulnerability assessment that is proposed here is deliberately evidence-based. It is meant to complement subsequent exploitation choices and not to take their place. This makes the test disciplined and the final report does not over-state what scanner can prove itself.

3.1 Vulnerability Assessment Methodology

```
hemanth@hemanth-virtual-machine:/var/www/html/attendize$ php artisan serve --host=0.0.0.0 --port=8000
Laravel development server started: http://0.0.0.0:8000
[Thu Apr  9 00:58:08 2026] PHP 7.4.33 Development Server (http://0.0.0.0:8000) started
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48940 Accepted
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48940 Closing
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48946 Accepted
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48946 Closing
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48954 Accepted
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48954 Closing
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48968 Accepted
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48968 Closing
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48970 Accepted
[Thu Apr  9 01:01:53 2026] 127.0.0.1:48970 [200]: GET /vendor/jquery/dist/jquery.min.js
```

Figure 11: Attendize server logs on Start up

The methodology of vulnerability assessment to be used in Attendize has a staged approach that proceeds to visibility to prioritization. The initial level is service-level reconnaissance. Prior to any web-specific interpretation being attempted, the target host must be located, contacted and verified as the appropriate assessment system. In this setup, the application could be accessed at 192.168.29.130:8000 in Kali Linux. That mere fact influences the remainder of the evaluation since that defines the host, the path into the web server and the network environment where findings of low severity against the host are discoverable. Here, the assessment sets some baseline data like whether the host is responding consistently, what service port is open and what initial headers or banners are shown.

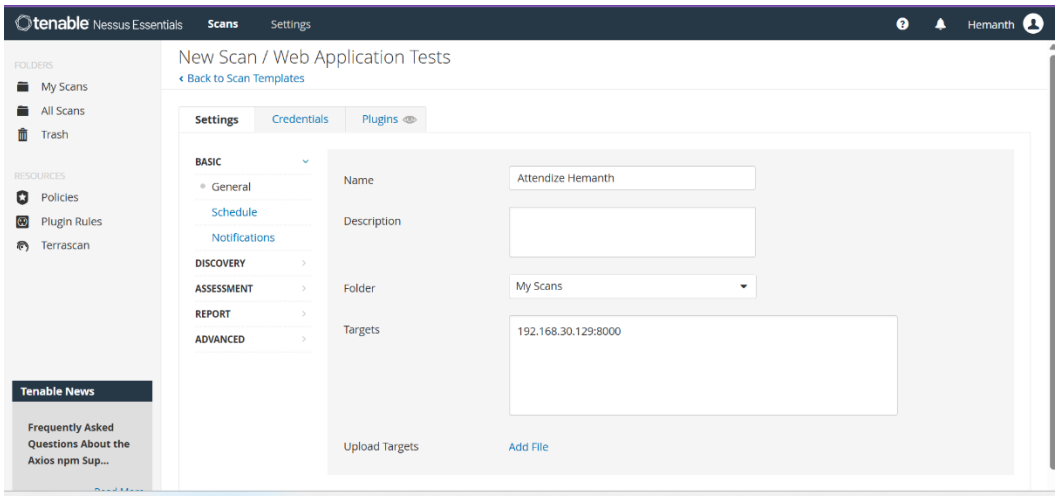


Figure 12: Nessus Scan

The second step is the host and server vulnerability discovery by Nessus Essentials. Nessus can be effectively used to emphasize the kind of bottom-up results which web-only scanners tend to ignore as being within their reach: version disclosure, robots.txt exposure, HTTP methods, timestamp behavior and service identification. This information is helpful to this target, since it lets the tester know how much the server will divulge prior to any intensive manual interaction. Though most of such findings are either informational or low severity, they are still part of attack planning and should not be ruled out. A robots.txt record or a permitted HTTP method, even a version string can assist in cutting down on the number of guesses that an attacker needs to make. The latest release notes and documentation of Tenable still indicate that Nessus is meant to consolidate precisely such a host and configuration intelligence into a well-organized report format (Tenable, 2024; Tenable, 2026).

The third step is ZAP web application discovery. ZAP needs to be programmed to crawl the public event pages, login routes and any check out flows that can be easily accessed without using destructive requests. Mapping is the main objective. Missing security headers, vulnerable client-side libraries, mixed content issues, insecure forms and other web-specific issues can then be identified by passive scanning by ZAP. This is essential to Attendize since the software is used to deliver content via browsers which can be analyzed at response level even prior to in-depth manual testing. Combined crawling and passive analysis of this kind is becoming a part of modern dynamic testing workflows due to its capability to give early indicators of browser-layer protection and component hygiene (ZAP by Checkmarx, 2024a; ZAP by Checkmarx, 2024b).

Correlation and prioritization is the fourth step. This is where scan results cease to be raw outputs and they turn into an evaluation. The lack of a Content Security Policy header is more serious when the application is also based on an old JavaScript library. Disclosure of version is even more interesting when the application is known to rely on a legacy generation of framework. A low risk robots.txt problem is more topical when it refers to working or privileged routes. In this step, the tester categorizes the findings into groups like information disclosure, configuration weakness, outdated component risk, access control test targets and workflow-critical areas. This will avoid the usual pitfall of considering all alerts as equally valuable.

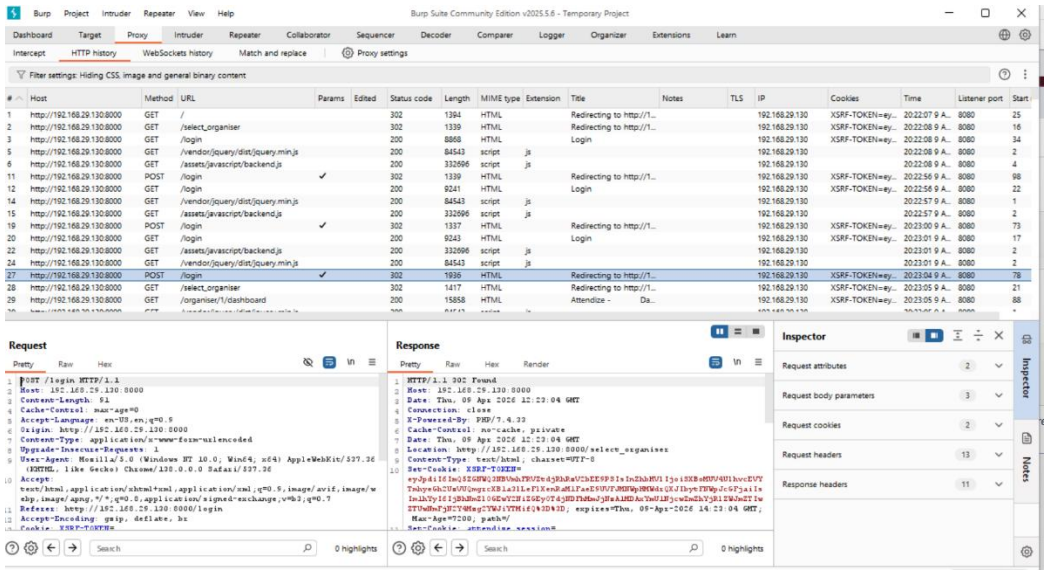


Figure 13: Burp Suite Request and Response testing

The fifth step is the manual validation planning. The final step of vulnerability assessment should be a shortlist of routes and behaviors to be investigated manually in Burp Suite. In the case of Attendize, the probably important areas are checkout validation, event creation, event objects reference, organizer actions, POST requests that change states, and routes with role boundaries or hidden assumptions. Recent literature greatly supports this shift of scanning to manual inspection. Automated scanners are useful at quickly spitting out more run-of-the-mill indicators but have less success with context-specific bugs like broken business logic, fragile state assumptions and subtle access control failures (Abdulghaffar, Elmrabit and Yousefi, 2023; Gandikota et al., 2023).

One last methodological consideration is the evidence handling. A description of every step of the vulnerability testing must be recorded in sufficient detail to assist in subsequent reporting: scan setup, target IP, tool name, match endpoint, alert type and interpretation. This proposal thus considers screenshots, route captures, scanner summaries and logs as a component of the methodology. The result of a scan out of context is highly aggregable, a scan result in context of a route, of a component or of an effect on operations is much more justifiable. In that regard, the vulnerability assessment methodology is not merely the process of locating vulnerabilities but also is the process of creating a chain of evidence which will aid in making risk-based decisions during the subsequent penetration testing.

Another methodology used is prioritization based on business relevance. Event browsing routes are significant, but checkout, organizer management and order creation routes are more crucial as they may directly impact sales, customer confidence and business sustainability. The assessment can be made more valuable to the system owner by assigning weight to scanner outputs based on business impact as opposed to merely severity labels. It is especially significant in the applications where the medium-level workflow problems can be more harmful in practice than the low-level disclosures, which are isolated.

4.0 Tools

The tools that were picked in this proposal have been chosen based on the real nature of the target and not because they are popular in isolation. Attendize is a hosted system, which is a web application, though it has visible services, headers, dependencies and workflow routes. Nessus Essentials and OWASP ZAP thus have different yet complementary purposes. Nessus and OWASP ZAP are used to support host and service visibility and dynamic web discovery, passive scanning and browser-facing analysis, respectively.

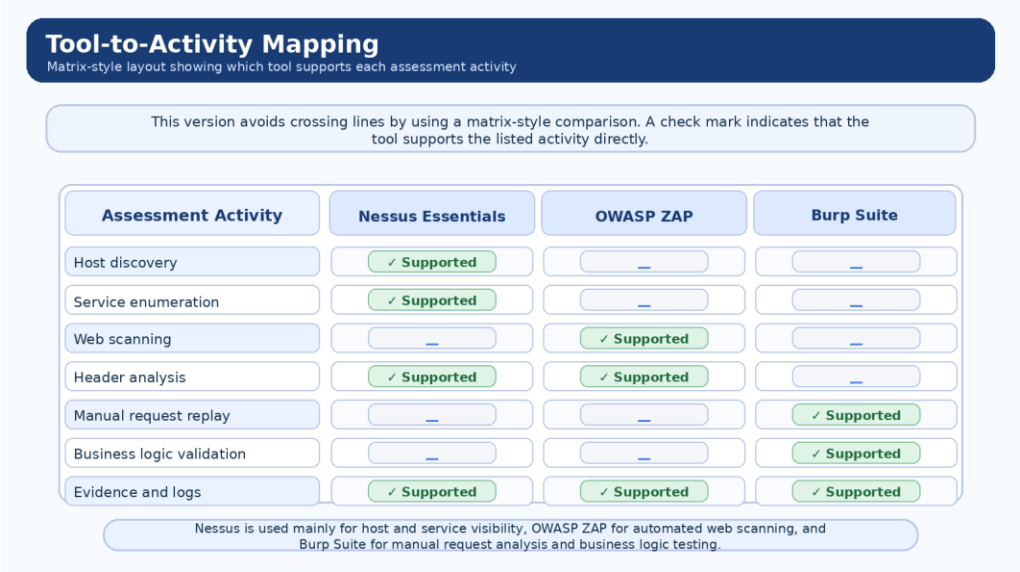


Figure 14: Tool-to-Activity Mapping

4.1 Nessus



Figure 15: Nessus Logo

The reason why Nessus Essentials has been chosen as one of the key vulnerability testing tools to be used in this proposal is the fact that it offers an organized host-based visibility that is used to supplement web-based testing. Although Attendize is technically a web application, it nonetheless runs on an operating system, which is accessed via a web server and offered by the services that are network relevant. Nessus is useful as it can determine service versions, protocol behavior, exposure of HTTP methods, information leakage and other environmental concerns that are not evident in a browser-only workflow. Tenable remains to position Nessus as an all-purpose vulnerability scanning platform that can be used to analyze network and system exposure (Tenable, 2024; Tenable, 2026).

In the case of a local Attendize deployment, the primary usefulness of Nessus will not be that it will inevitably find a spectacular remote code execution vulnerability. Instead, it has a value in its disciplined inventory and hardening evaluation. The generated report based on Nessus may indicate the presence of Apache and PHP exposing the identifying information, the presence of robots.txt revealing potentially useful paths, the server responding to the user of the timestamp requests, the presence of an unnecessary HTTP method, and the possibility of easy fingerprinting of the host. Hardening is often used in a mature enterprise to minimize this kind of leakage. Such details are often left in place in a lab or small deployment. That makes Nessus helpful as a scanning

tool as well as an educational aid to the knowledge of how small disclosures add to the total attack surface.

Nessus is appropriate as well since its output is in a format that can be readily converted into a proposal and eventually into a definitive exploitation report. It separates informational findings and low or higher severity findings, which will assist the tester against exaggeration. This is more so when it comes to an academic assignment where one of the mistakes that one is likely to make is mixing a lot of findings with a lot of serious findings. Nessus promotes a stricter definition: part of the results will indicate inadequate hardening; part will indicate visible configuration and only a portion of those will help in exploitation.

In general, Nessus has been right in this report as it provides a structured, plausible starting point of vulnerability assessment. It aids in finding version releases, low-level data leakage, observable configuration characteristics and other host-relevant problems that must be comprehended prior to more profound web assaults are endeavored.

The other practical benefit of Nessus Essentials is the ability to convey its findings in a way that can be easily understood by non-experts. Organizers or project supervisors might not be interested in the number of plugins, but they are familiar with the meaning of things like the server discloses its version, the site discloses robots.txt, or the host discloses timestamp requests. This renders Nessus results in an excellent compromise between technical evidence and discussion of risks on the level of management.

4.2 OWASP ZAP (Zed Attack Proxy)



Figure 16: OWASP ZAP Logo

The best automated web application assessment tool that can be used in this proposal is OWASP ZAP due to its strengths that fit perfectly well in the arrangement of the target. Attendize is a browser-based route-driven application that has public pages, form submissions, cookies, tokens and client-side resources. ZAP is created to monitor and analyses this type of behavior. It can act as a proxy, traverse reachable content, do passive scanning, and reveal web-specific vulnerabilities, including the absence of security-related headers, use of older JavaScript libraries and other response-related indicators. The official ZAP project remains focused on ease of onboarding and level of automation, with 2024 project releases indicating further improvement of scan rules and endpoint detection support (ZAP by Checkmarx, 2024a; ZAP by Checkmarx, 2024b).

In the case of Attendize, route discovery and passive analysis is the immediate worth of ZAP. ZAP has the capability to map event paths, forms, and static resources and response metadata before any form of aggressive request manipulation is executed. In a gray-box proposal, this comes in handy since the tester is aware of the overall nature of the application yet must still see how the application behaves during deployment. It is more systematic with a scanner that can spider and categories web content.

The second reason why ZAP is the right tool is that it discovers the kind of vulnerabilities that are often significant in a real web application but that are often underemphasized in student reports. Missing Content Security Policy, lack of X-Frame-Options, sub resource integrity and vulnerable browser-side libraries are not as exciting as injection but are significant since they undermine the defensive posture of the application. In case there is another defect, the effect of such omissions can be multiplied. When used correctly, ZAP is not a scanner, but an organizer of testing effort, directing the tester to areas that should be investigated manually in Burp Suite.

ZAP can also be useful in an academic context, as it creates evidence that can be accessed easily. The site tree, alert summary, risk levels and individual response snippets facilitate the explanation of why a web specific issue is important. That enhances the quality of the proposal and the subsequent exploitation report since the tester can demonstrate not only that an alert was raised, but also the location in the application where it was noticed.

4.3 Comparison Table between Nessus & OWASP ZAP

Table 1 compares the two assessment tools used in this proposal. The objective is not to identify a universal winner, but to show why both are required to cover different layers of the target.

Feature / Aspect	Nessus Essentials	OWASP ZAP
Primary focus	Host, network and service exposure assessment	Web application discovery and passive/active web analysis
Best fit for Attendize	Version disclosure, robots.txt, HTTP methods, low-level information leakage	Headers, vulnerable JS, page spidering, form and route visibility
Main strength	Structured severity reporting and environment-level findings	Web-context visibility and browser-facing issue detection
Main limitation	Limited understanding of business logic and route sequence	Needs manual validation for workflow, access control and state issues
Why included in this proposal	Provides system-level context for the Ubuntu host at 192.168.29.130	Provides web-layer context for Attendize routes and assets

Table 2 Comparison between Nessus & OWASP ZAP

5.0 Methodology

The most robust VAPT proposal is one that is grounded in a recognized methodology as opposed to an ad hoc process of tools being launched. The reason why methodology is important is that it will ensure that the assessment does not result in a checklist of irrelevant activities. It allows auditability of the report too: another tester can then be able to know how the report has been done and why certain paths of attack have been given priority. In this Attendize proposal, the methodology is deliberately designed based on the four phases of planning, discovery, attack and reporting model. This makes the engagement manageable and at the same time enables a wide enough coverage of a web application with multiple roles and workflows.

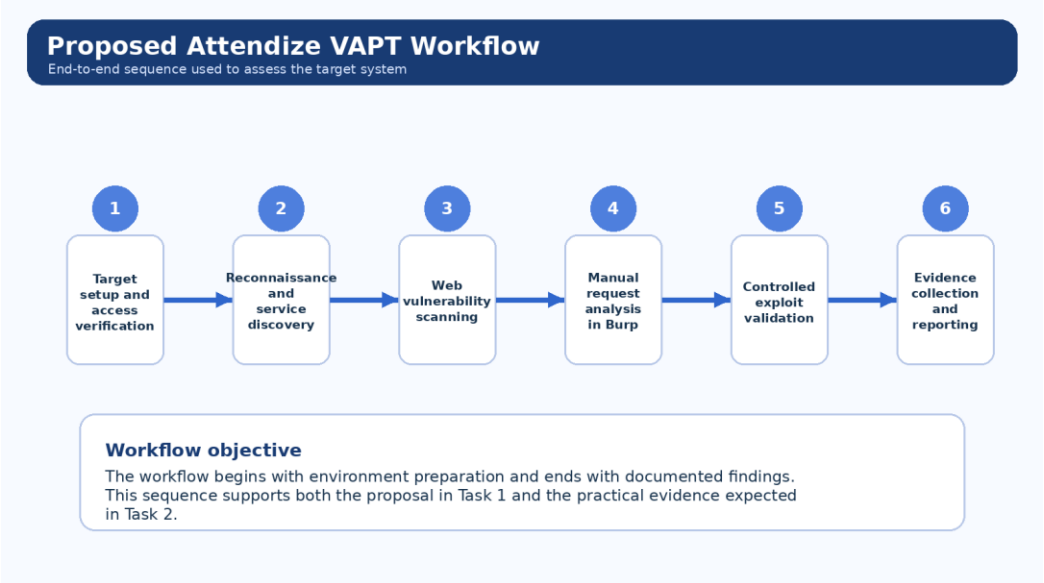


Figure 17: Proposed Attendize VAPT Workflow

The section title of this report, NIST 800-115 Methodology, reflects the sample structure, albeit in a modern manner. The initial NIST draft is older; nevertheless, the pattern of planning-discovery-attack-reporting related to NIST SP 800-115 is still applied in recent comparative and practical studies due to its practicality and simplicity of alignment with risk-based reporting (Akbar, Nur Isnaini and Putranto, 2025; Imtias et al., 2025). In the current Task 1 proposal, this structure is paired with the current web testing practice and the toolset utilized in the lab setup. Consequently, the methodology section not only is theoretically consistent with a recognized framework, but also is operationally consistent with Attendize, Ubuntu, Nessus Essentials, OWASP ZAP and Burp Suite.

5.1 NIST SP 800-115 Methodology

Planning is the initial step towards the methodology. Planning in this report is not merely about deciding of a target. It will encompass explanation of what Attendize is, why it should be tested, where it is hosted, what systems are involved and what the boundaries are. The local deployment applied in this proposal defined Ubuntu host as the authorized target, Kali Linux as the assessing workstation and 192.168.29.130:8000 as the primary application endpoint. It also defined the target class: a web-based ticketing and event management system with business-critical

processes revolving around event publication and order builders. Recent implementation research which nonetheless adheres to the NIST SP 800-115 model highlights that planning must comprise target acceptance, scope definition, testing objectives, tools and evidence requirements prior to the initiation of discovery (Akbar, Nur Isnaini and Putranto, 2025).

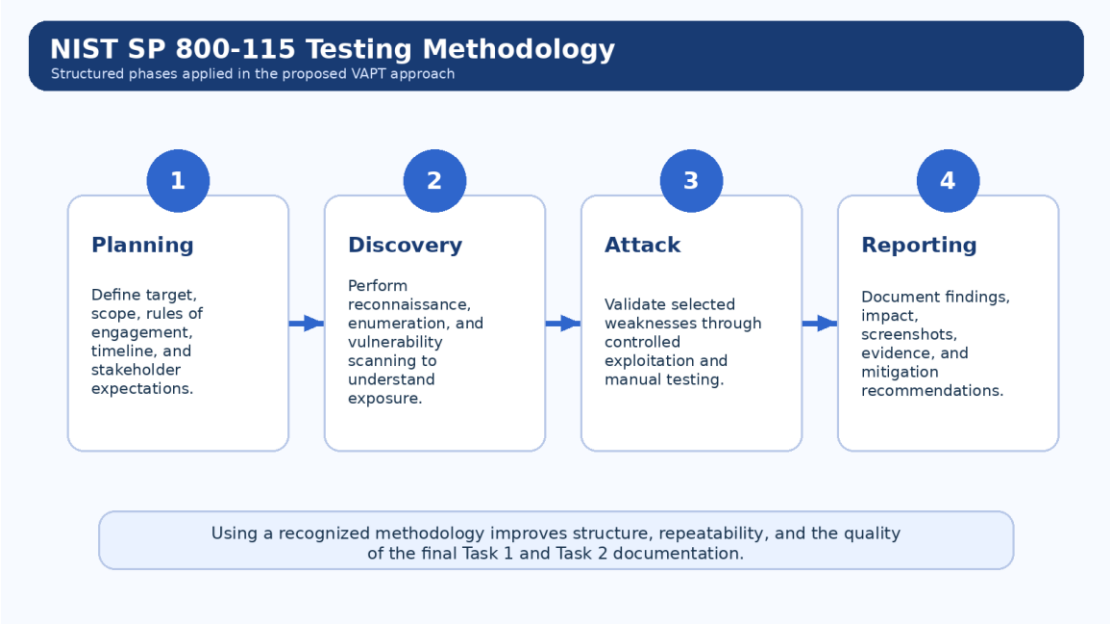


Figure 18: NIST SP 800-115 Testing Methodology

The second is the discovery stage. Host enumeration is paired with application mapping in a web application engagement to create discovery. In this target, the discovery phase will involve confirming the host and the services available, then to Nessus Essentials which will discover the environment level and OWASP ZAP will discover the web-route. Nessus provides server and exposure context, including version disclosures, HTTP method visibility and auxiliary leakage indicators. ZAP adds page crawling, asset discovery, passive issue detection and initial web-route comprehension. Results are then categorized as hardening weakness, supply-chain concern, likely manual test target or contextual low-value disclosure.

Third stage is attack. Attack in this proposal does not imply indiscriminate exploitation, but rather controlled manual verification of high-priority findings and high-risk workflow assumptions. Burp Suite is the major tool used in this phase since it enables the interception of requests, replacing and manipulation of the requests of HTTP. According to the Attendize architecture, the following areas of attack focus have been identified: multi-step checkout state,

event and order identifiers, state-changing POST requests, cookies, CSRF behavior and any action performed by the organizer which seems to receive object references on the client. This is the step where gray-box knowledge comes in especially handy. The tester is also aware of significant paths to attack, yet he or she interacts with the server through live HTTP interaction to discover its behavior under different circumstances.

Reporting is the fourth stage. Reporting must be a process of converting raw technical observations into security action. Because of this reason, the methodology makes every finding explain by at least five items: the endpoint or component that is affected, the source of evidence, the technical description, the business impact and the direction of the remediation. The latest literature confirms the practicality of the NIST-related four-stage pattern even in the context of web applications and its comparability to the OWASP- and PTES-based models due to the ease of conveying the model and its capability to modify the model to suit the latest tools (Akbar, Nur Isnaini and Putranto, 2025; Imtias et al., 2025).

Another advantage of the structure aligned with NIST is that it promotes the consistency between various tools. It is simple to dismiss Nessus, ZAP and Burp as dissimilar activities, without a defined methodology. This is solved by the planning-discovery-attack-reporting sequence which assigns each tool a particular role. The Nessus helps with discovery at the host level, ZAP helps with discovery at the web layer, Burp helps with attack validation, and the report combines the findings of all three. These are among the reasons why the framework is still useful despite being adapted to the practice of a web application nowadays.

6.0 Exploitation and Attack

The exploitation and attack aspect of this Task 1 proposal explains how the vulnerabilities discovered in the assessment would be verified in a controlled manner. The focus is on evidence of danger instead of devastating trade off. In the case of a business web application such as Attendize, the most valuable attack demonstrations tend to exhibit a trust or workflow failure as opposed to complete takeover. In case a checkout route can be crashed with a malformed state, in case an organizer’s action can be invoked without proper authorization, or in case a response includes missing browser protections that would magnify another vulnerability, then those are

security-relevant results. Thus, the following section dwells on a web-centric approach to manual attack based on request interception, replay and controlled tampering.

The idea behind the attack stage is not to distinguish between scanning and real testing as though scanners were worthless. Instead, it is to utilize scanner and discovery outputs to determine where manual effort is to be invested. In the suggested Attendize engagement, this would imply that Burp Suite should be used as the main exploiting tool, with OWASP ZAP being used as an auxiliary web proxy and testing tool. The nature of the target is very important in the design of the attack: a stateful application, which relies on cookies, CSRF controls, nested request parameters, object identifiers and sequence-sensitive order logic.

This step is thus intended to be a follow-up to the assessment procedure, and not a standing-alone exercise. The results of Nessus and ZAP influence the areas on which Burp Suite will be utilized the most, making sure that human work is dedicated to the routes and behaviors that have the greatest likelihood of being important.

6.1 Penetration Testing Attack Method.

The penetration testing attack approach that is proposed starts with browsing through Burp Suite as an authenticated and unauthenticated system. This is intentional as most web vulnerabilities manifest themselves when the application is pushed in a manner a typical user would push it. A static scan will not reveal the state of the validation step in the sense of whether it correctly stores state to be used later by an order-creation step, or whether an organizer-facing button will eventually execute a state-changing request based on an identifier supplied by a client. Thus, behavioral mapping is the first sub-stage of an attack: determine what requests are made when logging in, viewing events, selecting a ticket, and validating checkout and placing an order.

The second sub-stage is parameter and state analysis. In this, the tester examines what parameters are in each request, which are nested, which are empty, which are repeated in cookies or hidden fields, and which seem to make business decisions. The checkout process is particularly applicable in Attendize, as it holds buyer data, ticket-holder data, CSRF values and state changes in various paths. The tester is to find the normal sequence and then seek mismatch points: parameters that are supplied by validation, but not created, IDs that are supplied by the client, and

appear not to be generated by the server and route values that may seem to be optional, but actually influence logic and conditioning that may be based on assumptions that cannot be seen by the user.

Controlled request manipulation is the third sub-stage. The primary tool is Burp Repeater. Replay and validation can be performed to ensure reproducibility and then the validation can be done by changing one variable at a time to determine whether it is robust. Manipulations of candidates involve the alteration of event identifiers, the alteration or deletion of non-essential parameters, alteration of object identifiers, manipulation of state-related values, resending a request not in the anticipated sequence and testing whether state-changing endpoints reject malformed or incomplete data. This type of testing can be very useful in a ticketing application since core business risk can be due to mistaken assumptions about the order state, ownership continuity or session continuity, as opposed to due to classical injection.

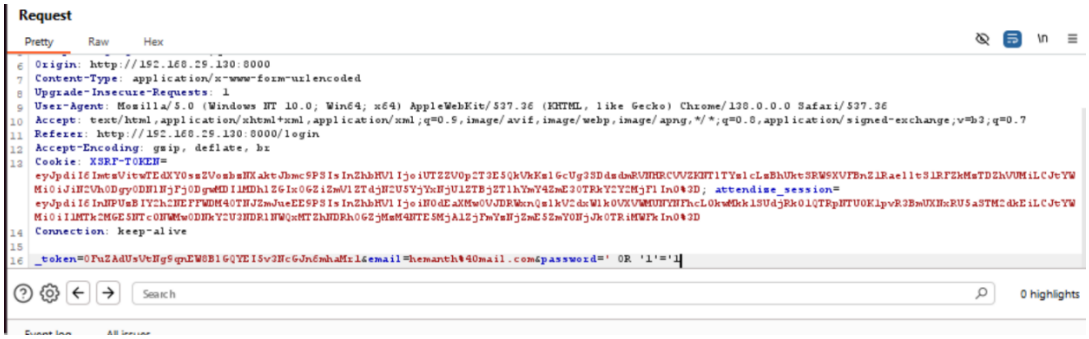


Figure 19: Simple Injection in Burp Suite (OR '1'='1)

The fourth sub-stage is the access-control-oriented validation. Where the application transmits object identifiers like event IDs, order IDs or organizer-identified resources, the tester ought to investigate either whether the server re-authenticates or it merely believes the client path. This is particularly significant considering that broken access control remains the most critical risk to web applications as per the current ranking of OWASP (OWASP, 2025b). The fifth sub-stage is browser-side, defensive-surface validation, where the absence of CSP, X-Frame-Options or outdated client-side components is related to actual attack implications. Lastly, resilience and error-handling testing must verify that a malformed or an incomplete request could terminate one of the core processes. Within business uses, a controlled exploit might merely indicate that a vital

workflow suffers a breakdown in unforeseen state. That is usually sufficient to demonstrate high availability and integrity risk.

The last advantage of this style of attack is that it will keep the interaction manageable. All the manipulations are limited, focused, and justifiable. Then it is simpler to halt at evidence-of-danger rather than getting lost in meaningless disturbance. This subject matter is significant in a scholarly setting since the aim of exploitation is not to make the lab a free-for-all, but to demonstrate the failure of the application. A good VAPT proposal is thus one that is technical but procedurally gagged.

6.2 Tools

The toolset of the exploitation in this proposal is specifically lightweight and web-oriented. The primary manual validation tool used is Burp Suite as it is best suited to the task of intercepting, replaying and controlling workflows of the HTTP protocol. OWASP ZAP is still helpful in the attack phase as a secondary proxy and check tool, especially in the context of verifying visibility of routes, passive alert environment and web-application architecture.

6.2.1 Burp Suite

The primary exploitation tool to use in this proposal to Attendize is Burp Suite since the primary attack surface is an HTTP-based and stateful one. The significant decisions made in the application are during requests: login, validation, creating orders, actions of the organizer and submissions of token-bearing forms. Burp Suite is tailored to bring these decisions to light. Its proxy gives the tester the ability to intercept traffic; HTTP history gives the ability to understand route sequencing; Repeater gives the ability to replay controlled; and request editing gives the ability to test accurately how the server will respond to assumptions being violated. That is necessary to conduct a gray-box test of a business application written in Laravel.

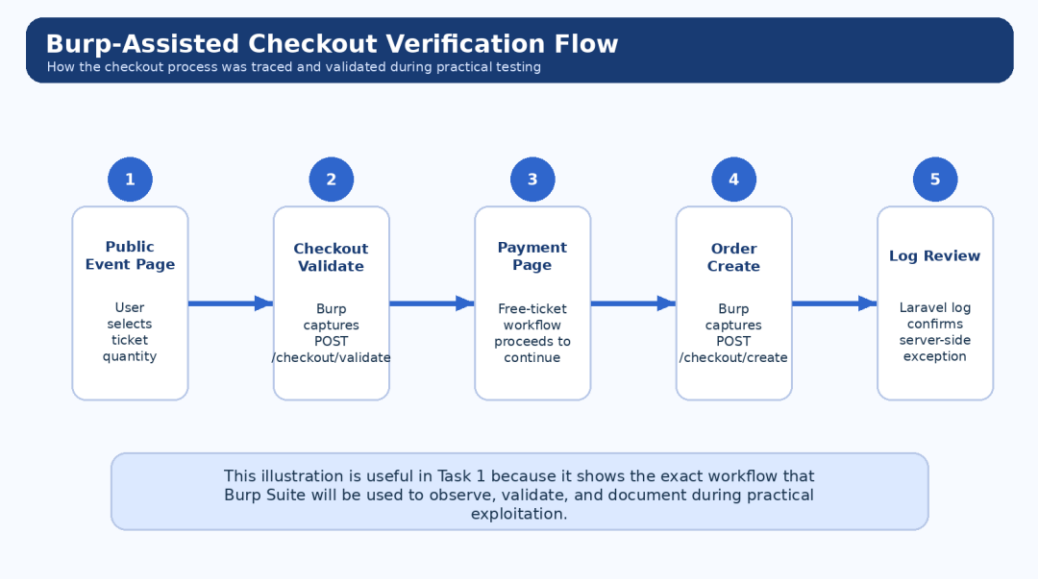


Figure 20: Burp-Assisted Checkout Verification Flow

Burp Suite is particularly robust when the goal is not to simply send payloads but to gain an insight into workflow logic. In Attendize, one transaction can consist of several routes with state associated with each. The execution of a regular browser can mask the architecture of that flow since the user only views pages and buttons. The request choreography is exposed through burp. This assists the tester in determining where validation is done, where state is constructed and which path ultimately does the consequential action. Knowledge is critical to testing business logic and server-side trust. That is, Burp Suite transforms a user path into a test worthy sequence of protocols.

The other reason why Burp Suite is suitable is that it fits well in the risk profile which was identified earlier in the report. Best investigated is broken access control, object reference handling, CSRF behavior and server-side validation assumptions, which can be analyzed best with the help of manual request inspection. Symptoms can be identified by automated scanners, but it is Burp that lets the tester demonstrate that a route is to be trusted by a parameter, that a state-changing request can be re-executed, that an empty field or a modified field can change behavior, and that the response varies with controlled tampering. This renders Burp especially useful when the exploitation stage of a ticketing application is involved where workflow misuse may not be detected in the unauthenticated scanning phase.

Burp also has high quality evidence. Repeater requests may be saved, compared and screen shot to report. This is significant since a VAPT report is as good as the evidence behind it. In case an error is identified during a manual test, a permission issue or a mismatched response, a Burp enables the tester to display what was sent, and what was received. This, when supplemented with logs of application runs on the server, creates a strong case of evidence. Academically, this would be very helpful since it aids in explaining and not merely asserting.

The extent to which Burp Suite can be applied in this proposal is, however, limited. The tool is not being selected to perform uncontrolled brute-force attacks, steal valuable real-world information or cause as much damage as possible. It is being selected to check the security of stateful web interactions under a controlled environment. This difference needs to be seen since the quality of the proposal will be based on the selection of the appropriate tool for the appropriate job. Burp is not meant to replace Nessus or ZAP, but it is the main manual exploitation tool that transforms scanner indications into defensible findings.

Due to these reasons, Burp Suite can be considered a worthy choice as the primary exploitation tool in this report. It directly aligns with the structure of the target, the nature of vulnerabilities that are most likely to be of interest and the evidence needs of quality VAPT engagement.

Burp Suite is also suitable for recording repeatability. In case a path breaks when a certain manipulated request is made, the tester can save the request, reread it to compare the result with the original flow. This renders the subsequent report more robust since the route of exploits is reproducible and technologically clear but not anecdotal.

6.2.2 OWASP ZAP

The OWASP ZAP also has a supporting role in the exploitation phase, although, in this report, its main role is in vulnerability assessment. This is because ZAP can both be an automated scanner and interactive proxy. Practically, this implies that it could be used to verify route behavior, passive alerts on the surface as the application is using it and give a second view on the structure of the application in combination with Burp Suite. This dual use is one of the reasons

why ZAP is still of great importance to web security workflows: it is easy to use when one needs to scan quickly, yet it is also flexible enough to help with the manual analysis.

In the case of the Attendize target, the exploitation-support value of ZAP is predominantly in three areas. First, it can be applied to verify whether a new route or asset is found or is publicly accessible and what it looks like when analyzed passively. Second, it can assist you to understand the security implications of browser facing problems in the form of missing headers or poor client-side elements. Third, it may be used as a cross-check in cases where the tester desires to know whether a route family or a group of assets has been mapped out exhaustively. According to the latest project documentation and updates, ZAP has been reinforcing these very endpoint-discovery and rule-maintenance features, which proves handy in case of testing changing web apps (ZAP by Checkmarx, 2024a; ZAP by Checkmarx, 2024b).

The tool must, however, not be considered as a substitute of Burp Suite in the ultimate test of concept phase. As a corroborating tool and a medium between evaluation and manual validation, it is most valuable in the exploitation section. Applied in this manner, ZAP assists in keeping efficiency and coverage without compromising the focus of the proposal on focused, route-specific manual testing.

6.2.3 Comparison Table Between Burp Suite and OWASP ZAP

Both Burp Suite and OWASP ZAP can proxy traffic, but in this proposal, they are assigned different exploitation roles. Burp is the primary manual validation tool, while ZAP supports route discovery and secondary verification.

Feature / Aspect	Burp Suite	OWASP ZAP
Primary role in Task 1	Manual interception, Repeater-based request replay and parameter/state testing	Spidering, passive alerting and supporting web proxy analysis
Best use against Attendize	Checkout validation, order creation, cookies, CSRF and route sequence testing	Header verification, route mapping and browser-facing issue confirmation

Evidence quality	High-quality request/response proof for reports	Strong route and alert summaries for supporting evidence
Main limitation	More manual effort required	Less precise than Burp for final proof-of-concept workflow abuse
Reason for inclusion	Best-fit exploitation tool for a stateful Laravel web application	Complements Burp and strengthens web assessment coverage

Table 3 Comparison between Burp Suite & OWASP ZAP

7.0 Conclusion

This report has given a comprehensive Task 1 VAPT proposal specific to Attendize ticketing and event management platform installed in a controlled Ubuntu-based lab setting. The report has demonstrated that Attendize is an appropriate target of a web application in terms of web application analysis and penetration testing exercise by adopting the structure of the provided sample but substituting the content with the purpose-specific analysis. It has a combination of public pages, account functionality, transactional checkout processes, organizer functionality and dependency-based functionality thus becoming a practical case study in web security evaluation today.

The proposal outlined the target system, its purpose, the operating environment and the software/services used. It explained a gray-box penetration testing strategy, the most realistic option since the tester has knowledge of the broad stack but must find, map and test the application using live HTTP behavior. It also demonstrated the reasons behind the necessity of both Nessus Essentials and OWASP ZAP, which are similar but different, and why Burp Suite is the most suitable primary exploitation tool towards the later part of the assessment.

Comprehensively, the given approach to VAPT is suitable, justifiable and detailed enough to complete Task 1. It is based on the real implementation, coordinated to the tools that were applied in the lab, as well as underpinned by the recent authoritative references that were published after 2022. More importantly, it approaches the target as a real business system with confidentiality, integrity and availability concerns and not a set of generic vulnerability names. This makes the proposal practically useful and a solid base to the subsequent Task 2 exploitation report.

8.0 References

Abdulghaffar, K., Elmrabit, N. and Yousefi, M. (2023) 'Enhancing Web Application Security through Automated Penetration Testing with Multiple Vulnerability Scanners', Computers, 12(11), p. 235. Available at: https://doi.org/10.3390/computers12110235.

Akbar, I., Nur Isnaini, K. and Putranto, B.D. (2025) 'Penetration Testing Through NIST SP 800-115 and OWASP TOP 10 With Risk Analysis Using CVSS on the XY Diskominfo Website', Journal of Innovation Information Technology and Application (JINITA), 7(2), pp. 314-326. Available at: https://doi.org/10.35970/jinita.v7i2.2907.

Altulaihan, E.A. et al. (2023) 'A Survey on Web Application Penetration Testing', Electronics, 12(5), p. 1229. Available at: https://doi.org/10.3390/electronics12051229.

Attendize (2024) Attendize. Available at: https://github.com/Attendize/Attendize.

Gandikota, P.S.S.K. et al. (2023) 'Web Application Security through Comprehensive Vulnerability Assessment', Procedia Computer Science, 230, pp. 168-182. Available at: https://doi.org/10.1016/j.procs.2023.12.072.

Imtias, M.B., Umam, K., Mustofa, H. and Subowo, M.H. (2025) 'Comparative Analysis of Penetration Testing Frameworks: OWASP, PTES, and NIST SP 800-115 for Detecting Web Application Vulnerabilities', Journal of Applied Informatics and Computing, 9(6), pp. 3689-3696. Available at: https://doi.org/10.30871/jaic.v9i6.9846.

Laravel (2026) Release Notes. Available at: https://laravel.com/docs/13.x/releases.

OWASP (2025a) OWASP Top 10:2025. Available at: https://owasp.org/Top10/2025/.

OWASP (2025b) A01:2025 Broken Access Control. Available at: https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/.

OWASP (2025c) A03:2025 Software Supply Chain Failures. Available at: https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/.

Tenable (2024) Nessus 2024 Release Notes. Available at: https://docs.tenable.com/release-notes/Content/nessus/2024.htm.

Tenable (2026) Nessus Documentation. Available at: https://docs.tenable.com/Nessus.htm.

ZAP by Checkmarx (2024a) ZAP Updates - January 2024. Available at: https://www.zaproxy.org/blog/2024-02-02-zap-updates-january-2024/.

ZAP by Checkmarx (2024b) Powering Up DAST with ZAP and Noir. Available at: https://www.zaproxy.org/blog/2024-11-11-powering-up-dast-with-zap-and-noir/.